



Informe Laboratorio: Branch and Bound para TSP

Profesor = Cristian Javier Cubillos Araya

Estudiante = Aidan Tavish Reid Gonzalez

Fecha = 28-05-2026

1. Introducción

En este laboratorio abordamos el **Problema del Viajante de Comercio (TSP)**, que consiste en encontrar la ruta más eficiente para visitar un grupo de ciudades (sin repetir ninguna) y regresar al punto de partida. Para resolverlo, implementamos el método de **Ramificación y Poda (Branch & Bound)** y evaluamos cómo se comporta el algoritmo al usar distintas estrategias de exploración y límites de búsqueda. Además, desarrollamos una función para exportar todo el árbol de decisiones a archivos **JSON** y **Graphviz DOT**, lo que nos permite visualizar de manera gráfica cómo va trabajando el algoritmo paso a paso.

2. Configuración Experimental

La matriz de costos utilizada fue:

Desde/ Hacia	A	B	C	D	E
A	∞	14	4	10	20
B	14	∞	7	8	12
C	4	5	∞	16	3
D	11	7	16	∞	2
e	18	10	4	2	∞

3. Implementación

La solución fue desarrollada en Python utilizando:

- Estrategia FIFO
- Estrategia LIFO
- Estrategia Best-First

Cada nodo almacenó:

- ID secuencial
- Camino parcial
- Cota inferior
- Estado
- Matriz reducida residual

Los árboles generados fueron exportados en:

- JSON
- DOT
- PNG

4. Resultados Experimentales

Se generaron los siguientes escenarios:

- Best-First + reducción robusta
- LIFO + reducción robusta
- Best-First + cota débil
- Best-First + modificación $C \rightarrow E = 99$

Cada escenario produjo:

- - Árbol JSON
 - Grafo DOT
 - Imagen PNG

Pregunta 4.1 — Anatomía de la Poda y la Incumbente Temprana

Se ejecutó el algoritmo utilizando las estrategias LIFO y Best-First.

En la estrategia LIFO, la primera solución completa encontrada fue el nodo 10.

El camino encontrado fue:

$0 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$

y el costo obtenido fue:

[57]

Más adelante, el algoritmo encontró mejores soluciones hasta llegar al costo óptimo de 30.

En el árbol Best-First se observa que algunos nodos fueron creados pero no expandidos porque su cota era peor que la incumbente encontrada.

Por ejemplo, el nodo 12 tenía:

[bound = 36]

mientras que la incumbente ya era:

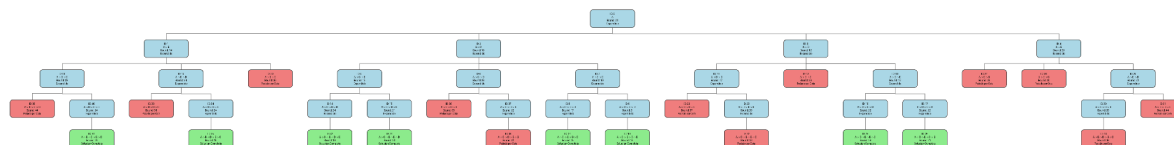
[30]

Por esta razón, el nodo fue podado y no se siguió expandiendo.

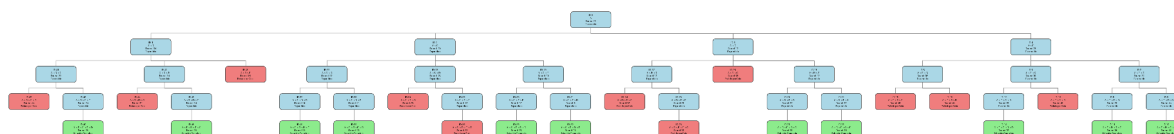
Esto demuestra que encontrar una incumbente temprana ayuda a reducir el espacio de búsqueda y evita explorar ramas innecesarias.

Fotos De Los Arboles

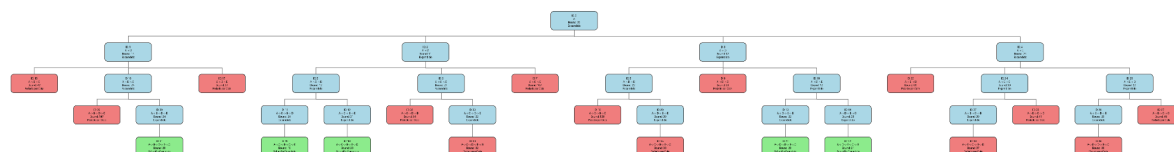
BEST-FIRST



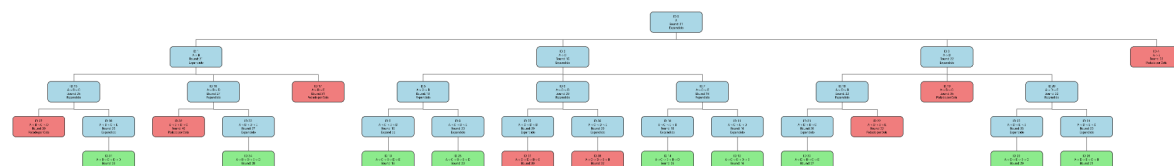
LIFO (Depth-First)



Arbol CE99



Arbol Weak



Pregunta 4.2 — Sensibilidad de la Función de Acotación

Se comparó el algoritmo utilizando dos funciones de acotación:

Reducción completa de filas y columnas.

Cota débil basada en mínimos locales.

Comparación de nodos generados

Método	Nodos Generados
Cota robusta	44
Cota débil	39

Aunque ambos métodos encontraron la misma solución óptima con costo 30, la estructura de los árboles fue diferente.

La cota robusta utiliza reducción de matrices para calcular cotas más precisas, mientras que la cota débil solamente considera mínimos locales y pierde información global del problema.

En el árbol con cota débil se observan ramas que continúan expandiéndose aunque finalmente no mejoran la solución.

Por ejemplo, el camino:

$0 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 4$

alcanzó una solución con costo:

[45]

a pesar de que ya existía una incumbente mejor de:

[30]

Tabla Comparativa

Método de cota	Nodos generados	Nodos expandidos	Nodos podados
Reduction	44	34	12
Weak	39	32	8

Esto demuestra que una función de acotación más fuerte permite podar antes y controlar mejor el crecimiento del árbol de búsqueda.

Pregunta 4.3 — Efecto Espejismo y Resiliencia Topológica

En este experimento se modificó el costo entre C y E, cambiando el valor de:

$3 \rightarrow 99$

Esto generó cambios importantes en el árbol de búsqueda.

En el caso original, el nodo:

$[0,2,4]$

tenía una cota de:

10

y permitía encontrar rápidamente la mejor ruta:

$[0,2,4,3,1,0]$

con costo:

30

Después de modificar el costo, ese mismo nodo pasó a tener:

$\text{bound}=107$

por lo que fue podado inmediatamente:

$107 \geq 32$

Debido a esto, el algoritmo dejó de explorar esa rama y comenzó a priorizar otras rutas.

La nueva mejor solución encontrada fue:

[0,1,3,4,2,0]

con costo:

32

Esto demuestra que cambiar una sola conexión puede modificar completamente la estructura del árbol y las decisiones de poda del algoritmo.

5.- Enlace Al Repositorio Github

https://github.com/DoctorKong21/Guia_Laboratorio_Branch_and_Bound_TSP

6. Conclusiones

Los resultados obtenidos demuestran experimentalmente el impacto crítico de las funciones de acotación en la eficiencia del algoritmo Branch & Bound.

La reducción completa de matrices permitió disminuir significativamente la cantidad de nodos expandidos respecto de la cota ingenua, evidenciando la importancia de incorporar restricciones globales de conectividad en el cálculo de cotas inferiores.

Asimismo, las diferencias observadas entre las estrategias LIFO y Best-First muestran cómo el orden de exploración altera drásticamente la morfología del árbol y la aparición temprana de incumbentes.

Finalmente, la modificación localizada de una única arista crítica permitió observar fenómenos de resiliencia topológica y propagación diferida de costos dentro del espacio de búsqueda.